

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT
DELIVERABLE 2 – SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Field	Entry
Project Title	BookNest – Online Bookstore Ordering and Inventory Management System
Group Number	Team 6
Version	1.0
Course	UE24CS341A – Software Engineering
Technology Stack	MERN (MongoDB, Express.js, React.js, Node.js)

1. Introduction

1.1 Purpose

This document specifies the software requirements for BookNest, an online bookstore ordering and inventory management system. It covers version 1.0 of the system and defines the functional, non-functional and security requirements that the customer-facing storefront and the administrative back-office must satisfy.

1.2 Intended Audience and Reading Suggestions

This SRS is intended for the development team (all four members), the course instructor/evaluator, and the project stakeholder (Pageturner Books). Developers should read Sections 3–7 closely before design; the evaluator and stakeholder may focus on Sections 2, 5 and 6 to confirm scope and behaviour.

1.3 Product Scope

BookNest allows a customer to browse and search a catalog of books, view book details, manage a shopping cart, place an order with payment, and track order status. Store staff (administrator) can manage the book catalog and stock, and view basic sales/operational reports. The system replaces Pageturner Books' current phone-and-counter-based ordering process with a self-service web application.

1.4 References

- PES University – Software Engineering Project Guidelines 2026 (Jackfruit Mini-Project).
- PES University – SE Level 3 Student Submission Guide & Exemplar.
- IEEE SRS template (as supplied with the course guidelines).

2. Overall Description

2.1 Product Perspective

BookNest is a new, self-contained web application (not a follow-on to an existing system). It is a three-tier web application: a React.js single-page front end, an Express.js/Node.js REST API, and a MongoDB data store, integrated with an external/mock payment service.

2.2 Product Functions

- Customer account registration and login.
- Book catalog browsing, search and filtering.
- Shopping cart management.

- Order placement, payment and order tracking.
- Administrator catalog and inventory management.
- Administrator operational reporting.

2.3 User Classes and Characteristics

User class	Privileges	Characteristics
Customer	Browse, search, cart, order, pay, track own orders	Non-technical, occasional to frequent use, primarily mobile/desktop browser
Administrator	Manage catalog, stock and view reports	Store staff at Pageturner Books; higher privilege; trained user
Payment Service	Authorize payment and return success/failure	External/mock system, not a human user

2.4 Operating Environment

Server: Node.js runtime with Express.js, deployed to a cloud host (e.g. Render/Railway) or containerized with Docker. Database: MongoDB (Atlas or self-hosted). Client: any evergreen desktop or mobile web browser (Chrome, Firefox, Edge, Safari) accessing the React front end over HTTPS.

2.5 Design and Implementation Constraints

- Must use the MERN stack (MongoDB, Express.js, React.js, Node.js) as the chosen technology stack.
- Must follow Agile methodology with visible sprint/backlog activity in GitHub Projects or Jira.
- Must use Git/GitHub for version control, with CI/CD, code review and security validation evidence.
- Payment integration may use a sandbox/mock payment provider (e.g. Stripe/Razorpay test mode) given student project constraints.

2.6 Assumptions and Dependencies

- Assumes availability of a free-tier MongoDB Atlas cluster and a sandbox payment gateway account for the duration of the project.
- Assumes Pageturner Books' catalog data (title, author, price, stock) will be seeded/mock by the team rather than sourced from a live inventory feed.
- Depends on third-party npm packages for authentication (e.g. JWT) and payment SDK integration.

3. External Interface Requirements

3.1 User Interfaces

A responsive React.js web UI with: a customer-facing storefront (catalog, book detail, cart, checkout, order tracking, login/register) and an admin console (catalog management, stock, reports). Common elements: persistent navigation bar, cart icon with item count, form validation messages, and loading/error states on all data-fetching screens.

3.2 Software Interfaces

Component	Interface
Front end ↔ Back end	REST API over HTTPS, JSON payloads
Back end ↔ Database	MongoDB via Mongoose ODM
Back end ↔ Payment Service	REST/SDK call to sandbox payment gateway (e.g. Stripe/Razorpay test mode)

3.3 Communications Interfaces

All client-server communication uses HTTPS. Authenticated requests carry a JWT bearer token in the Authorization header. JSON is the sole message format for the REST API.

4. Analysis Models

Use Case Diagram

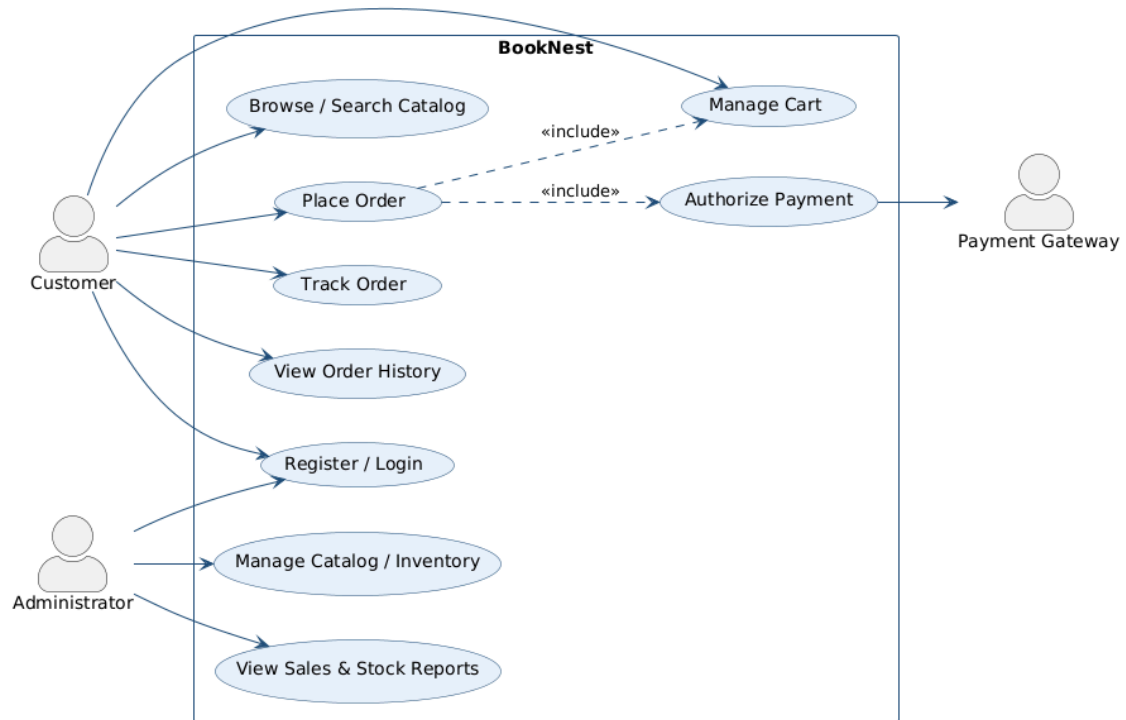


Figure 4.1 – Use Case diagram: Customer, Administrator and Payment Gateway.

Entity-Relationship Diagram

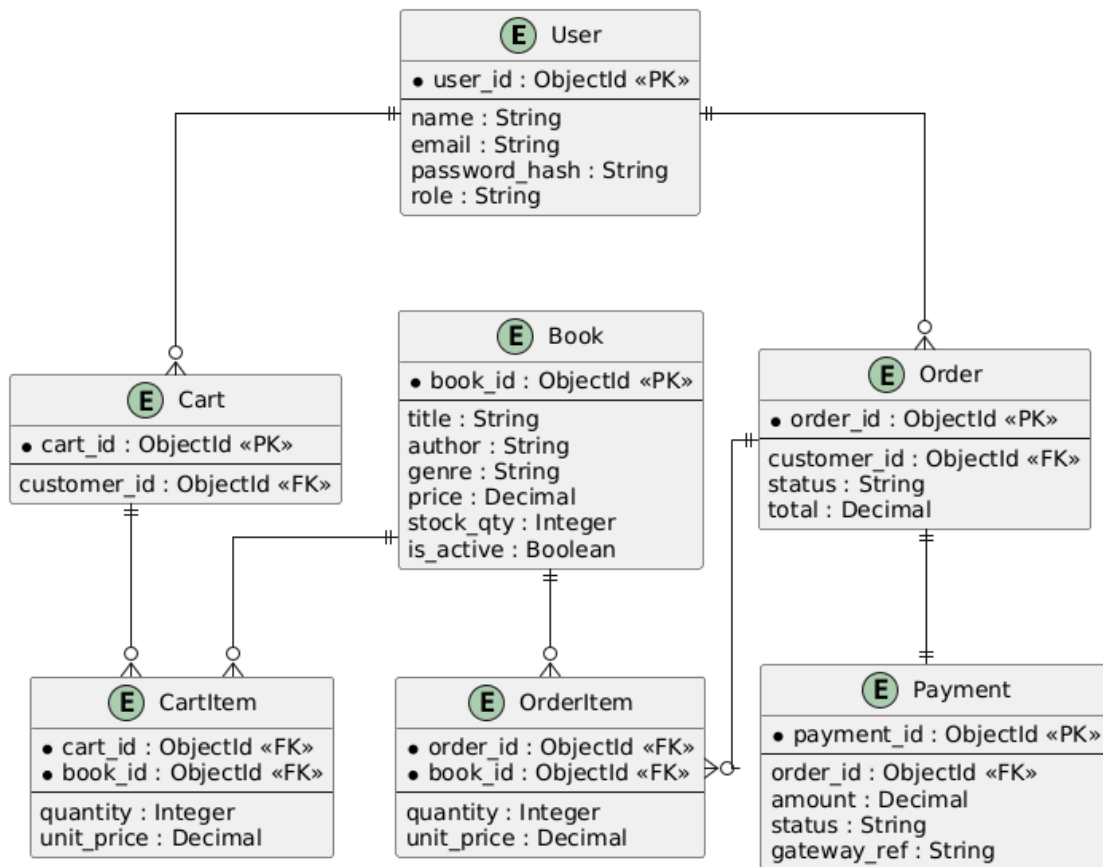


Figure 4.2 – Domain/ER model: User, Book, Cart, CartItem, Order, OrderItem, Payment.

The full class diagram (with operations) and remaining UML views (sequence, activity, component, deployment) are provided in the Design deliverable, Section 3.3.

5. System Features (Functional Requirements)

ID	Requirement	Priority	Verification
FR-001	The system shall allow a customer to create an account using a unique email address and password.	High	Functional test
FR-002	The system shall allow an authenticated customer to log in and receive a session token.	High	Functional test
FR-003	The system shall allow any visitor to browse and search the book catalog by title, author or genre.	High	Functional test
FR-004	The system shall allow a customer to filter catalog results by price range and availability.	Medium	Functional test
FR-005	The system shall allow a customer to add an available book to their cart and update its quantity.	High	Functional test
FR-006	The system shall calculate the cart total from item price and quantity.	High	Unit + system test
FR-007	The system shall create an order only after payment has been authorized successfully.	High	Negative + positive test
FR-008	The system shall assign a unique order number to every successfully created order.	High	Integration test
FR-009	The system shall allow a customer to view the current status of their order (RECEIVED, PROCESSING, SHIPPED, DELIVERED).	High	System test
FR-010	The system shall allow a customer to view their past order history.	Medium	Functional test

ID	Requirement	Priority	Verification
FR-011	The administrator shall be able to add, edit and disable book listings, including stock quantity.	High	Functional test
FR-012	The system shall record payment success or failure against the corresponding order.	High	Integration test
FR-013	The administrator shall be able to view a sales summary and a low-stock report.	Medium	Functional test

Requirement-writing rule: each requirement is uniquely tagged and is concise, complete, unambiguous, verifiable and necessary, per the supplied SRS template.

6. Other Nonfunctional Requirements

6.1 Performance, Availability, Usability, Reliability, Maintainability, Testability

ID	Category	Measurable requirement	Verification
NFR-001	Performance	95% of catalog search requests shall receive a response within 2 seconds under the defined test load.	Performance test
NFR-002	Availability	The application shall be available at least 99% during the defined operating window, excluding planned maintenance.	Operational review
NFR-003	Usability	A new customer shall be able to place a basic order in no more than 5 primary screens.	Usability walkthrough
NFR-004	Reliability	An order shall not be created twice when a customer retries after a payment timeout.	Negative/integration test
NFR-005	Maintainability	Business logic shall be separated into identifiable Express modules/services with automated unit tests.	Code review
NFR-006	Testability	Core services shall expose testable interfaces without requiring the live payment gateway (mockable adapter).	Unit/integration test

6.2 Security Requirements

ID	Security requirement	Validation approach
SEC-001	Passwords shall be hashed (e.g. bcrypt) and never stored in plaintext.	Code/configuration review
SEC-002	Role-based access shall prevent customers from accessing administrator endpoints.	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied fields before processing.	Negative/security tests
SEC-004	Authenticated APIs shall reject expired or invalid JWTs.	Security test
SEC-005	Sensitive payment information (e.g. card numbers) shall not be stored by the application; only tokens/references from the payment gateway are retained.	Design review
SEC-006	Security-relevant events such as failed login and unauthorized access attempts shall be logged.	System test

6.3 Business Rules / Domain Rules

- Only books marked available/in-stock can be added to a cart or order.
- A customer cannot modify an order once it has moved past RECEIVED status.
- Only an administrator can disable a book listing or adjust stock manually.
- An order must contain at least one valid item.
- A payment failure shall not result in a confirmed order or a stock deduction.
- Stock quantity is decremented only after payment is confirmed successful.

7. Use Cases

Use Case ID	Use Case	Primary actor	Precondition	Outcome
UC-01	Register / Login	Customer	Application available	Authenticated session

Use Case ID	Use Case	Primary actor	Precondition	Outcome
UC-02	Browse / Search Catalog	Customer	None	Matching books displayed
UC-03	Manage Cart	Customer	Logged in	Cart updated with items/quantities
UC-04	Place Order	Customer	Logged in; valid cart	Order created after payment
UC-05	Track Order	Customer	Order exists	Current status displayed
UC-06	Manage Catalog / Inventory	Administrator	Admin authenticated	Catalog/stock updated
UC-07	View Reports	Administrator	Admin authenticated	Sales/low-stock report displayed

8. Validation Specification – Initial Planning

At this stage, validation planning is focused on how requirements will eventually be validated; this is not the final test report.

Requirement category	Planned validation	Example
Functional	Black-box functional tests	Valid/invalid order placement
Security	Authentication/authorization + input validation testing	Customer attempts admin endpoint
Performance	Load/response-time testing	Concurrent catalog search/order requests
Usability	Task-based walkthrough	New user places an order in ≤5 screens
Reliability	Failure/retry scenarios	Payment timeout and retry
Unit	Automated unit tests + coverage	Cart total calculation, stock decrement logic

9. Project Plan (Work Breakdown)

Work package	Tasks	Owner	Est. effort (person-days)	Output
WP1	Requirements + use cases	All	4	SRS
WP2	Architecture + UML	M1/M2	4	Architecture/Design
WP3	Customer auth module	M1	6	Working feature
WP4	Catalog/search module	M2	6	Working feature
WP5	Cart/order/payment module	M3	7	Working feature
WP6	Admin inventory/reporting module	M4	7	Working feature
WP7	Testing + CI/CD + security	All	6	Test/validation evidence
WP8	Final validation + report	All	4	Final report

10. Tool Stack

Activity	Tool	Evidence to submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects / Jira	Backlog, user stories, sprint board

Activity	Tool	Evidence to submit
CI/CD	GitHub Actions / Jenkins	Pipeline run/log
Containerization	Docker	Dockerfile + build/run evidence
Testing	Jest / Supertest / React Testing Library	Test results
Static analysis	SonarQube or similar	Analysis report
API testing	Postman	Collection/results
Design	Draw.io / StarUML	UML diagrams

Appendix A: Glossary

Term	Definition
Catalog	The full set of book listings available for browsing/search.
Cart	A customer's temporary, pre-checkout collection of selected books.
Order	A confirmed, paid request to purchase one or more books.
SKU	Stock keeping unit — unique identifier for a book listing.
RBAC	Role-Based Access Control — restricting actions by user role (customer vs. admin).
JWT	JSON Web Token — used to authenticate API requests.

Appendix B: Field Layouts

Sample field layout for the Book collection:

Field	Length	Data type	Description	Mandatory
book_id	24	ObjectId	Unique book identifier	Y
title	150	String	Book title	Y
author	100	String	Author name	Y
genre	50	String	Genre/category	N
price	10,2	Decimal	Selling price	Y
stock_qty	6	Integer	Current stock available	Y
is_active	1	Boolean	Whether listing is enabled	Y

Sample field layout for the Order collection:

Field	Length	Data type	Description	Mandatory
order_id	24	ObjectId	Unique order identifier	Y
customer_id	24	ObjectId	Reference to customer	Y
status	20	Enum/String	RECEIVED / PROCESSING / SHIPPED / DELIVERED	Y
total	10,2	Decimal	Order total amount	Y
payment_status	20	Enum/String	SUCCESS / FAILED / PENDING	Y

Appendix C: Requirement Traceability Matrix (initial)

Architecture, design and code/test references will be completed once the Design deliverable is finalized; requirement IDs and descriptions are locked at this stage.

Req ID	Brief description	Architecture ref	Design ref	Code/file ref	Test Case ID	System Test ID
FR-001	Customer registration	TBD	TBD	TBD	TBD	TBD
FR-007	Order created after payment success	TBD	TBD	TBD	TBD	TBD
FR-011	Admin catalog/stock management	TBD	TBD	TBD	TBD	TBD

Req ID	Brief description	Architecture ref	Design ref	Code/file ref	Test Case ID	System Test ID
SEC-002	RBAC on admin endpoints	TBD	TBD	TBD	TBD	TBD
NFR-001	Catalog search response time	TBD	TBD	TBD	TBD	TBD

Note: replace all <placeholder> fields and TBD entries as the design and implementation stages proceed. This SRS should be revised (versioned) if requirements change after review.